Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №6
По дисциплине «Математические основы интеллектуальных систем»
За 3 семестр
Тема: «Проверка бинарных деревьев на изоморфность»

Выполнил:
Студент ФЭИС
Группы ИИ-29
Игнатюк Д.А.

Проверил:
Козинский А.А.

Брест, 2026

Лабораторная работа №6

«Проверка бинарных деревьев на изоморфность»

Цель работы: освоить алгоритмы обхода бинарных деревьев и реализовать программу на языке С для проверки двух деревьев на изоморфность, а также определения их высоты и сбалансированности.

Ход работы

Код

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
// ПРЕДСТАВЛЕНИЕ ДЕРЕВА
typedef struct {
    int left;
    int right;
} Node;
typedef struct {
    int n;              // число вершин
    int root;           // корень
    Node* nodes;        // массив вершин (индексы 1..n)
} Tree;
// Инициализация дерева
static void init_tree(Tree* t, int n) {
    t->n = n;
    t->root = 0;
    t->nodes = (Node*)calloc(n + 1, sizeof(Node));
}
// Освобождение
static void free_tree(Tree* t) {
    if (t->nodes) {
        free(t->nodes);
        t->nodes = NULL;
    }
    t->n = 0;
    t->root = 0;
}
// Чтение дерева из файла
static int read_tree(const char* fname, Tree* t) {
    FILE* f = fopen(fname, "r");
    if (!f) { printf("Oshibka\n"); return 0; }
    char line[256];
    int n = 0;
    int header_ok = 0;
    while (fgets(line, sizeof(line), f)) {
        if (sscanf(line, "%d", &n) == 1) { header_ok = 1; break; }
    }
    if (!header_ok || n <= 0) {
        printf("Oshibka\n");
        fclose(f);
        return 0;
    }
    init_tree(t, n);
    int first_mentioned = 0;
    while (fgets(line, sizeof(line), f)) {
        int u = 0, v = 0;
```

```
        int got = sscanf(line, "%d %d", &u, &v);
        if (got < 2) continue;
        // Проверки на корректность
        if (u < 1 || u > n || v < 1 || v > n || u == v) {
            printf("Oshibka\n");
            fclose(f);
            free_tree(t); // Освобождаем память при ошибке!
            return 0;
        }
        if (first_mentioned == 0) { first_mentioned = u; }
        // Вставка ребёнка
        if (t->nodes[u].left == 0) {
            t->nodes[u].left = v;
        }
        else if (t->nodes[u].right == 0) {
            t->nodes[u].right = v;
        }
        else {
            printf("Oshibka (bolshe 2 detey)\n");
            fclose(f);
            free_tree(t); // Освобождаем память при ошибке!
            return 0;
        }
    }
    fclose(f);

    t->root = first_mentioned ? first_mentioned : 1;
    return 1;
}
// ИТЕРАТИВНЫЕ ОБХОДЫ
static void preorder(const Tree* t, int* res, int* len) {
    int* stack = (int*)malloc((t->n + 1) * sizeof(int));
    int top = 0;
    *len = 0;
    if (t->root == 0) { free(stack); return; }
    stack[top++] = t->root;
    while (top > 0) {
        int u = stack[--top];
        res[(*len)++] = u;
        if (t->nodes[u].right) stack[top++] = t->nodes[u].right;
        if (t->nodes[u].left)  stack[top++] = t->nodes[u].left;
    }
    free(stack);
}
static void inorder(const Tree* t, int* res, int* len) {
    int* stack = (int*)malloc((t->n + 1) * sizeof(int));
    int top = 0;
    *len = 0;
    int curr = t->root;
    while (curr != 0 || top > 0) {
        while (curr != 0) {
            stack[top++] = curr;
            curr = t->nodes[curr].left;
        }
        curr = stack[--top];
        res[(*len)++] = curr;
        curr = t->nodes[curr].right;
    }
    free(stack);
```

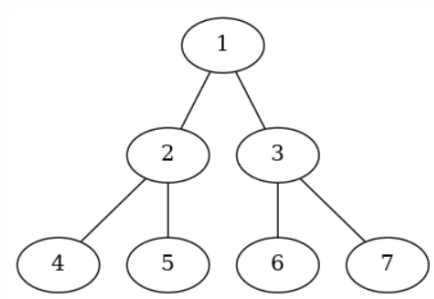
```
}
static void postorder(const Tree* t, int* res, int* len) {
    int* stack1 = (int*)malloc((t->n + 1) * sizeof(int));
    int* stack2 = (int*)malloc((t->n + 1) * sizeof(int));
    int top1 = 0, top2 = 0;
    *len = 0;
    if (t->root == 0) { free(stack1); free(stack2); return; }
    stack1[top1++] = t->root;
    while (top1 > 0) {
        int u = stack1[--top1];
        stack2[top2++] = u;
        if (t->nodes[u].left)  stack1[top1++] = t->nodes[u].left;
        if (t->nodes[u].right) stack1[top1++] = t->nodes[u].right;
    }
    while (top2 > 0) {
        res[(*len)++] = stack2[--top2];
    }
    free(stack1); free(stack2);
}
// ПРОВЕРКА ИЗОМОРФИЗМА
static int are_isomorphic(const Tree* t1, const Tree* t2) {
    if (t1->n != t2->n) return 0;
    int* s1 = (int*)malloc((t1->n + 1) * sizeof(int));
    int* s2 = (int*)malloc((t1->n + 1) * sizeof(int));
    int top = 0;
    s1[top] = t1->root;
    s2[top] = t2->root;
    top++;
    int iso = 1;
    while (top > 0) {
        top--;
        int u1 = s1[top];
        int u2 = s2[top];
        if (u1 == 0 && u2 == 0) continue;
        if (u1 == 0 || u2 == 0) { iso = 0; break; }
        s1[top] = t1->nodes[u1].left;
        s2[top] = t2->nodes[u2].left;
        top++;
        s1[top] = t1->nodes[u1].right;
        s2[top] = t2->nodes[u2].right;
        top++;
    }
    free(s1); free(s2);
    return iso;
}
// ВЫСОТА И СБАЛАНСИРОВАННОСТЬ
static int tree_height(const Tree* t, int u) {
    if (u == 0) return 0;
    int max_h = 0;
    int* sv = (int*)malloc((t->n + 1) * sizeof(int));
    int* sd = (int*)malloc((t->n + 1) * sizeof(int));
    int top = 0;
    sv[top] = u;
    sd[top] = 1;
    top++;
    while (top > 0) {
        top--;
        int v = sv[top];
        int d = sd[top];
```

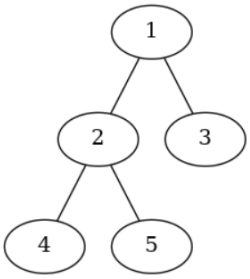
```
        if (d > max_h) max_h = d;
        if (t->nodes[v].left) { sv[top] = t->nodes[v].left; sd[top] = d
+ 1; top++; }
        if (t->nodes[v].right) { sv[top] = t->nodes[v].right; sd[top] =
d + 1; top++; }
    }
    free(sv); free(sd);
    return max_h;
}
static int is_balanced(const Tree* t) {
    for (int i = 1; i <= t->n; i++) {
        int hl = tree_height(t, t->nodes[i].left);
        int hr = tree_height(t, t->nodes[i].right);
        int diff = hl - hr;
        if (diff < 0) diff = -diff;
        if (diff > 1) return 0;
    }
    return 1;
}
int main(int argc, char** argv) {
    if (argc < 3) {
        printf("Oshibka (nuzhno 2 fayla)\n");
        return 1;
    }
    Tree t1, t2;
    if (!read_tree(argv[1], &t1)) return 1;
    // Если второе дерево не прочиталось, освобождаем первое
    if (!read_tree(argv[2], &t2)) { free_tree(&t1); return 1; }
    printf("=== Derevo 1 (%s) ===\n", argv[1]);
    printf("Vershin: %d, Koren: %d\n", t1.n, t1.root);
    int* res = (int*)malloc((t1.n + 1) * sizeof(int));
    int len;
    preorder(&t1, res, &len);
    printf("Pryamoy: "); for (int i = 0; i < len; i++) printf("%d ",
res[i]); printf("\n");
    inorder(&t1, res, &len);
    printf("Simmetrichnyy: "); for (int i = 0; i < len; i++) printf("%d
", res[i]); printf("\n");
    postorder(&t1, res, &len);
    printf("Obratnyy: "); for (int i = 0; i < len; i++) printf("%d ",
res[i]); printf("\n");
    int h1 = tree_height(&t1, t1.root);
    printf("Vysota: %d\n", h1);
    printf("Sbalansirovano: %s\n\n", is_balanced(&t1) ? "da" : "net");
    printf("=== Derevo 2 (%s) ===\n", argv[2]);
    printf("Vershin: %d, Koren: %d\n", t2.n, t2.root);
    preorder(&t2, res, &len);
    printf("Pryamoy: "); for (int i = 0; i < len; i++) printf("%d ",
res[i]); printf("\n");
    inorder(&t2, res, &len);
    printf("Simmetrichnyy: "); for (int i = 0; i < len; i++) printf("%d
", res[i]); printf("\n");
    postorder(&t2, res, &len);
    printf("Obratnyy: "); for (int i = 0; i < len; i++) printf("%d ",
res[i]); printf("\n");
    int h2 = tree_height(&t2, t2.root);
    printf("Vysota: %d\n", h2);
    printf("Sbalansirovano: %s\n\n", is_balanced(&t2) ? "da" : "net");
    printf("=== PROVERKA NA IZOMORFNOST ===\n");
```

```
    if (are_isomorphic(&t1, &t2)) {
        printf("Derevya izomorfny.\n");
    }
    else {
        printf("Derevya NE izomorfny.\n");
    }
    free(res);
    free_tree(&t1);
    free_tree(&t2);
    return 0;
}
```

Проверки:

Тест01+11



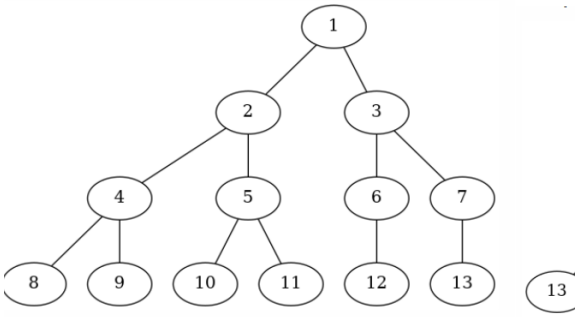


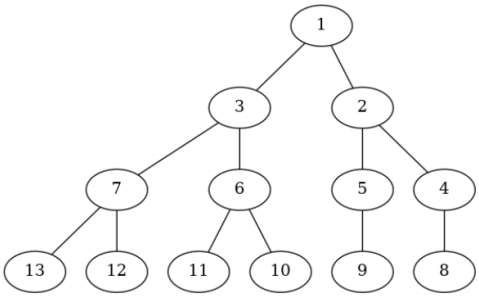

```
=== Derevo 1 (tests/test01.txt) ===
Vershin: 7, Koren: 1
Pryamoy: 1 2 4 5 3 6 7
Simmetrichnyy: 4 2 5 1 6 3 7
Obratnyy: 4 5 2 6 7 3 1
Vysota: 3
Sbalansirovano: da

=== Derevo 2 (tests/test11.txt) ===
Vershin: 5, Koren: 1
Pryamoy: 1 2 4 5 3
Simmetrichnyy: 4 2 5 1 3
Obratnyy: 4 5 2 3 1
Vysota: 3
Sbalansirovano: da

=== PROVERKA NA IZOMORFNOST ===
Derevya NE izomorfny.
```

Тест02+12



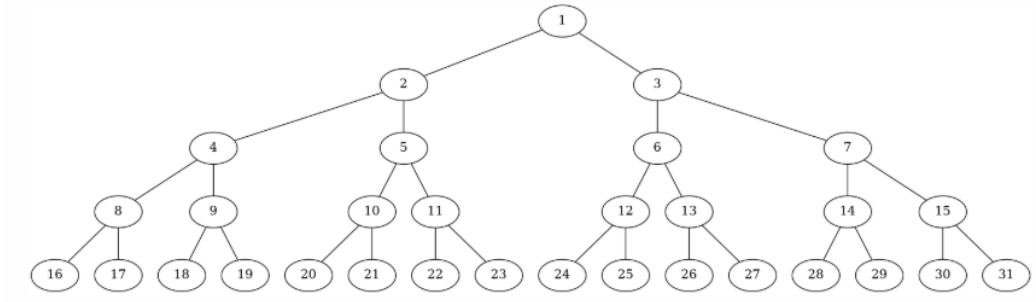


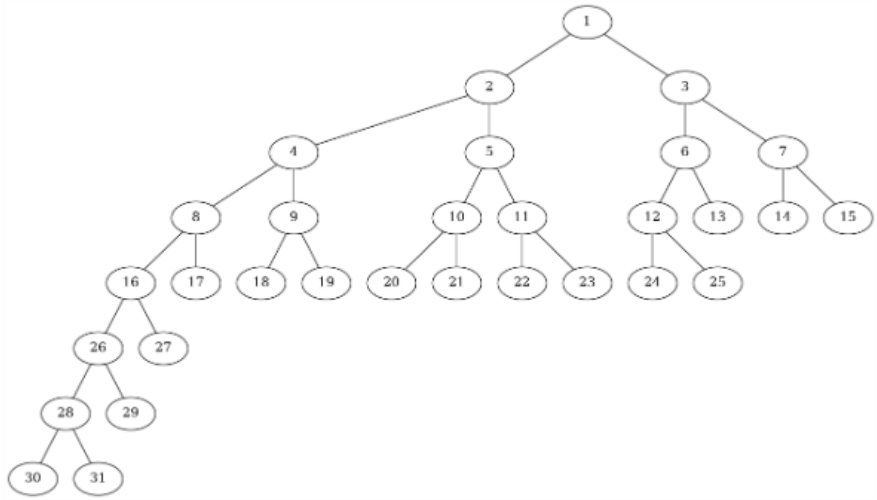

```
=== Derevo 1 (tests/test12.txt) ===
Vershin: 13, Koren: 1
Pryamoy: 1 3 7 13 12 6 11 10 2 5 9 4 8
Simmetrichnyy: 13 7 12 3 11 6 10 1 9 5 2 8 4
Obratnyy: 13 12 7 11 10 6 3 9 5 8 4 2 1
Vysota: 4
Sbalansirovano: da

=== Derevo 2 (tests/test02.txt) ===
Vershin: 13, Koren: 1
Pryamoy: 1 2 4 8 9 5 10 11 3 6 12 7 13
Simmetrichnyy: 8 4 9 2 10 5 11 1 12 6 3 13 7
Obratnyy: 8 9 4 10 11 5 2 12 6 13 7 3 1
Vysota: 4
Sbalansirovano: da

=== PROVERKA NA IZOMORFNOST ===
Derevya izomorfny.
```

Тест03+13



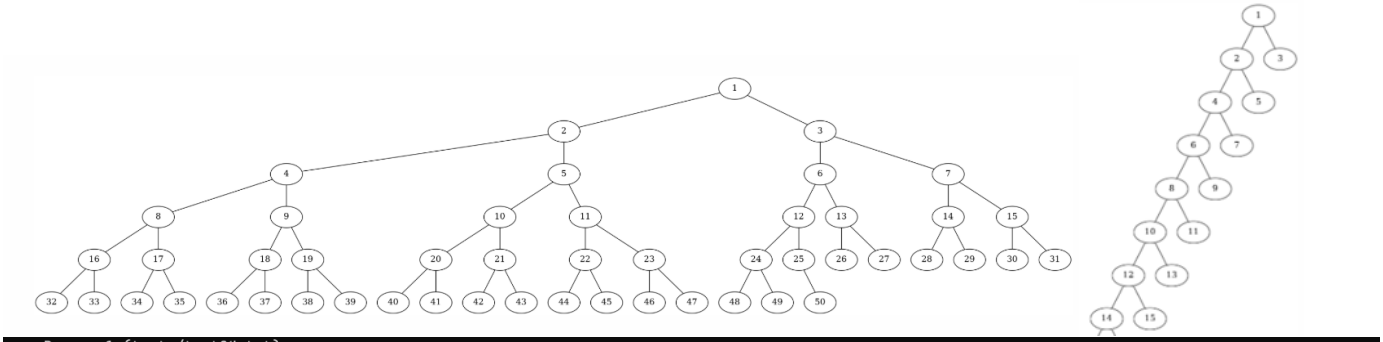


```
=== Derevo 1 (tests/test03.txt) ===
Vershin: 31, Koren: 1
Pryamoy: 1 2 4 8 16 17 9 18 19 5 10 20 21 11 22 23 3 6 12 24 25 13 26 27 7 14 28 29 15 30 31
Simmetrichnyy: 16 8 17 4 18 9 19 2 20 10 21 5 22 11 23 1 24 12 25 6 26 13 27 3 28 14 29 7 30 15 31
Obratnyy: 16 17 8 18 19 9 4 20 21 10 22 23 11 5 2 24 25 12 26 27 13 6 28 29 14 30 31 15 7 3 1
Vysota: 5
Sbalansirovano: da

=== Derevo 2 (tests/test13.txt) ===
Vershin: 31, Koren: 1
Pryamoy: 1 2 4 8 16 26 28 30 31 29 27 17 9 18 19 5 10 20 21 11 22 23 3 6 12 24 25 13 7 14 15
Simmetrichnyy: 30 28 31 26 29 16 27 8 17 4 18 9 19 2 20 10 21 5 22 11 23 1 24 12 25 6 13 3 14 7 15
Obratnyy: 30 31 28 29 26 27 16 17 8 18 19 9 4 20 21 10 22 23 11 5 2 24 25 12 13 6 14 15 7 3 1
Vysota: 8
Sbalansirovano: net

=== PROVERKA NA IZOMORFNOST ===
Derevya NE izomorfny.
```

Тест04+14

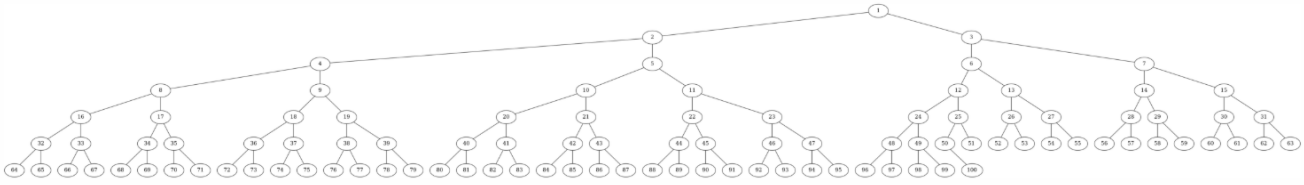


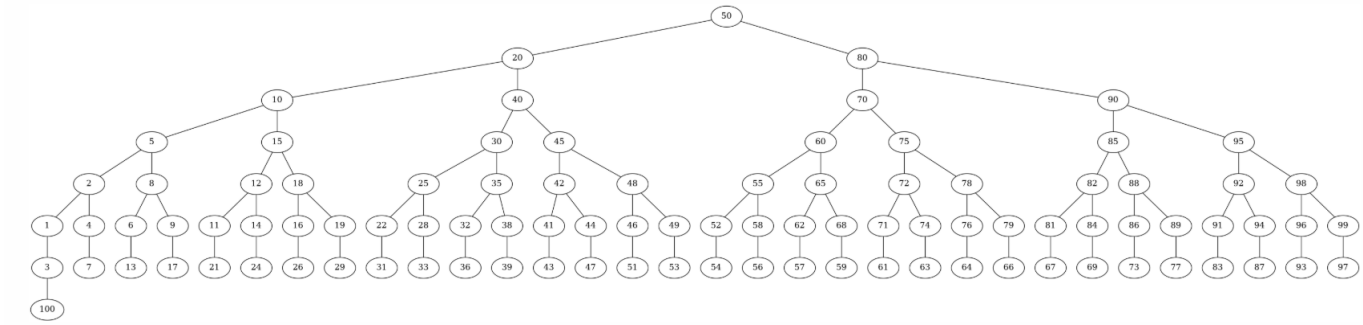
```
=== Derevo 1 (tests/test04.txt) ===
Vershin: 50, Koren: 1
Pryamoy: 1 2 4 8 16 32 33 17 34 35 9 18 36 37 19 38 39 5 10 20 40 41 21 42 43 11 22 44 45 23 46 47 3 6 12 24 48 49 25 50 13 26 27 7 14 28 29 15 30 31
Simmetrichnyy: 32 16 33 8 34 17 35 4 36 18 37 9 38 19 39 2 40 20 41 10 42 21 43 5 44 22 45 11 46 23 47 1 48 24 49 12 50 25 6 26 13 27 3 28 14 29 7 30 15 31
Obratnyy: 32 33 16 34 35 17 8 36 37 18 38 39 19 9 4 40 41 20 42 43 21 10 44 45 22 46 47 23 11 5 2 48 49 24 50 25 12 26 27 13 6 28 29 14 30 31 15 7 3 1
Vysota: 6
Sbalansirovano: da

=== Derevo 2 (tests/test14.txt) ===
Vershin: 50, Koren: 1
Pryamoy: 1 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 49 47 45 43 41 39 37 35 33 31 29 27 25 23 21 19 17 15 13 11 9 7 5 3
Simmetrichnyy: 50 48 46 49 44 47 42 45 40 43 38 41 36 39 34 37 32 35 30 33 28 31 26 29 24 27 22 25 20 23 18 21 16 19 14 17 12 15 10 13 8 11 6 9 4 7 2 5 1 3
Obratnyy: 50 48 49 46 47 44 45 42 43 40 41 38 39 36 37 34 35 32 33 30 31 28 29 26 27 24 25 22 23 20 21 18 19 16 17 14 15 12 13 10 11 8 9 6 7 4 5 2 3 1
Vysota: 26
Sbalansirovano: net

=== PROVERKA NA IZOMORFNOST ===
Derevya NE izomorfny.
```

Тест05+15 (100вершин)






```
=== Derevo 1 (tests/test05.txt) ===
Vershin: 100, Koren: 1
Pryamoy: 1 2 4 8 16 32 64 65 33 66 67 17 34 68 69 35 70 71 9 18 36 72 73 37 74 75 19
21 42 84 85 43 86 87 11 22 44 88 89 45 90 91 23 46 92 93 47 94 95 3 6 12 24 48 96 97
 14 28 56 57 29 58 59 15 30 60 61 31 62 63
Simmetrichnyy: 64 32 65 16 66 33 67 8 68 34 69 17 70 35 71 4 72 36 73 18 74 37 75 9 7
0 84 42 85 21 86 43 87 5 88 44 89 22 90 45 91 11 92 46 93 23 94 47 95 1 96 48 97 24 9
55 3 56 28 57 14 58 29 59 7 60 30 61 15 62 31 63
Obratnyy: 64 65 32 66 67 33 16 68 69 34 70 71 35 17 8 72 73 36 74 75 37 18 76 77 38 7
 86 87 43 21 10 88 89 44 90 91 45 22 92 93 46 94 95 47 23 11 5 2 96 97 48 98 99 49 24
7 28 58 59 29 14 60 61 30 62 63 31 15 7 3 1
Vysota: 7
Sbalansirovano: da

=== Derevo 2 (tests/test15.txt) ===
Vershin: 100, Koren: 50
Pryamoy: 50 20 10 5 2 1 3 100 4 7 8 6 13 9 17 15 12 11 21 14 24 18 16 26 19 29 40 30
 47 48 46 51 49 53 80 70 60 55 52 54 58 56 65 62 57 68 59 75 72 71 61 74 63 78 76 64
 92 91 83 94 87 98 96 93 99 97
Simmetrichnyy: 100 3 1 2 7 4 5 13 6 8 17 9 10 21 11 12 24 14 15 26 16 18 29 19 20 31
 44 45 51 46 48 53 49 50 54 52 55 56 58 60 57 62 65 59 68 70 61 71 72 63 74 75 64 76
 90 83 91 92 87 94 95 93 96 98 97 99
Obratnyy: 100 3 1 7 4 2 13 6 17 9 8 5 21 11 24 14 12 26 16 29 19 18 15 10 31 22 33 28
3 49 48 45 40 20 54 52 56 58 55 57 62 59 68 65 60 61 71 63 74 72 64 76 66 79 78 75 70
4 92 93 96 97 99 98 95 90 80 50
Vysota: 8
Sbalansirovano: net
```

Тест06+16 (500 вершин)

```
=== Derevo 1 (tests/test06.txt) ===
Vershin: 500, Koren: 1
Pryamoy: 1 2 4 8 16 32 64 128 256 25
```

```
Vysota: 9
Sbalansirovano: da

=== PROVERKA NA IZOMORFNOST ===
Derevya izomorfny.
```

Тест07+17 (1000 вершин)

```
=== Derevo 1 (tests/test07.txt) ===
Vershin: 1000, Koren: 1
Pryamoy: 1 2 4 8 16 32 64 128 256 512
 33 66 132 264 528 529 265 530 531 13
44 545 273 546 547 137 274 548 549 27
```

```
Vysota: 10
Sbalansirovano: da

=== PROVERKA NA IZOMORFNOST ===
Derevya NE izomorfny.
```

Тест08+18 (10000 вершин)

```
=== Derevo 1 (tests/test08.txt) ===
Vershin: 10000, Koren: 1
Pryamoy: 1 2 4 8 16 32 64 128 256 512
```

```
Vysota: 14
Sbalansirovano: da

=== PROVERKA NA IZOMORFNOST ===
Derevya izomorfny.
```

Вывод: реализована программа на C для проверки бинарных деревьев на изоморфность, определения высоты и сбалансированности. Проверена на 8 парах тестов — от малых деревьев до критического теста на 10 000 вершин. Результаты совпали с теорией.